WEEK 03

Collections, loops and totals

This week: learn, practise, build

01 · Python

Collections, loops and totals
Understand the idea and trace examples.

02 · Practice

Practise the same Python concept.
Predict, change, diagnose and explain.

03 · Project

Multi-batch aggregation
Build, discuss and test with a peer.

One weekly component moves the final project forward.

Python foundations

Collections, loops and totals
Concept → trace a small example → explain its behavior

Lists, records and accumulation

A list holds records

[0] selects the first record.
["points"] selects its field.

The total has a lifetime

Reset before the loop.
Update inside; report after.

A count is not a sum

len(records) counts rows.
Adding points measures total points.

Trace a small list, then write a loop that also handles empty input.

Store many values, then process each one

List: an ordered collection

Use an index to select one item. A list may contain dictionaries.

Dictionary: named fields

Read a value by its key, such as row["count"]. Keys describe the data.

Loop: repeat a block

for visits each item. Start a total once, then add each contribution inside the loop.

Understand the behavior before changing the example.

A loop keeps its running state

1

Before the loop

total = 0
Initialize once

2

For each item

Read fields
Calculate and add

3

After the loop

Report the final total
Empty list leaves zero

Read the diagram, then trace the same steps in code.

Syntax to read and reuse

Python / syntax	Meaning	When to use it
[item, item]	List: ordered items	Store several records
{"key": value}	Dictionary: named fields	Describe one record
for item in items:	Visit each item	Process a collection
total += value	Add to the running total	Accumulate contributions

Python Basics · sections 05, 06 · use only when you need a lookup.

Create, count and read a plain list

```
counts = [2, 4, 1]
print(len(counts))
print(counts[0])
print(counts[1])
empty = []
print(len(empty))
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Update, append and remove an item

```
01 counts = [2, 4]
02 counts[1] = 3
03 print(counts)
04 counts.append(1)
05 print(counts)
06 last = counts.pop()
07 print(last, counts)
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Visit a plain list with a loop

```
01 counts = [2, 4, 1]
02 for count in counts:
03     print(count)
04 print("Finished")
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Accumulate numbers before records

```
counts = [2, 4, 1]
total = 0
for count in counts:
    total += count
    print("Running:", total)
print("Final:", total)
assert total == 7
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Read one dictionary before a list of dictionaries

```
01 record = {"name": "A", "points": 2}
02 print(record["name"])
03 print(record["points"])
04 record["points"] = 3
05 print(record["points"])
```

1. Predict each printed result first.
2. Run this small example on its own.
3. Explain the operation before combining it.

Learn the single operation before the combined example.

Read fields and accumulate a total

```
records = [{"name": "A", "count": 2},
           {"name": "B", "count": 3}]
print(records[0]["name"])
total = 0
for row in records:
    total += row["count"]
    print(row["name"], total)
print("total", total)
```

1. Expected: A; A 2; B 5; total 5.
2. The accumulator starts before the loop.
3. Try an empty list: what still runs?

Predict → execute → explain the value at each step.

Lists contain items; dictionaries contain fields

1. **List** — Ordered items. A position selects one item; indexing starts at zero.
2. **Dictionary** — Named key/value pairs. A key selects one field.
3. **Value** — The selected value can be used in an expression or another statement.

A list position and a dictionary key select different things.

Read a dictionary value by key

```
line = {"batch_id": "T01",
        "portions_per_container": 3, "containers": 4}
print(line["batch_id"])
print(line["containers"])
planned = line["portions_per_container"]
planned *= line["containers"]
print("Planned portions:", planned)
assert planned == 12
```

1. Predict the ID, count and labelled portion estimate
2. Keys retrieve named values
3. A wrong key spelling raises KeyError

The estimate combines two fields from the same record.

for: visit each item in a list

```
lines = [
    {"batch_id": "T01", "containers": 4},
    {"batch_id": "T02", "containers": 2},
]
for line in lines:
    print(line["batch_id"], line["containers"])
print("Lines:", len(lines))
assert len(lines) == 2
```

1. Write the output in execution order.
2. The indented print runs for each item.
3. The final print runs after the loop.

The loop body runs once per item; len() counts items.

Trace the accumulator after each contribution

```
contributions = [12, 8]
total = 0
for contribution in contributions:
    before = total
    total += contribution
    print(before, "+", contribution, "=", total)
assert total == 20
```

1. Trace the first contribution from the initial zero
2. Use that running total for the second contribution
3. The previous total is part of the next calculation

Initialize once, add each contribution, then report.

An empty list: zero loop iterations

```
contributions = []
planned_portions = 0
for contribution in contributions:
    planned_portions += contribution
print("Planned portions:", planned_portions)
assert planned_portions == 0
```

1. How often does the body execute?
2. Which value remains afterward?
3. Initialize before the loop, even for [].

An empty list skips the loop body and keeps the initial value.

Hands-on practice

Week_03.ipynb · Session 2
Predict → run → change → explain

Your turn: practise the same Python skills

P1–P2 · Easy

Use the approach hints. Write the code yourself in a blank cell.

P3 · Core build

Implement a complete small program. Write expected results and assertions.

P4–P5 · Optional

Apply the skill to a new case, then attempt a harder challenge if time permits.

Write your own code, predict the result, then test and explain it.

Check your work before asking for help

A complete implementation

Write the whole operation. Do not stop at one changed character.

Independent expected results

Calculate a normal case and an edge case before running your program.

A peer can explain it

Swap driver and reviewer. Explain one line and one test.

Read list/dictionary values and trace an accumulator, including empty input.

Project studio

Multi-batch aggregation
Short design briefing → most time for your work

The people’s task this week

1

Someone needs help

A pickup can contain more than one kind of food.

2

This week, you will

Read each food line and add its containers and planned portions.

3

A useful result

The coordinator can read the totals without mixing lines, containers and portions.

Use the supplied starting files. Focus on one useful result and check it.

Test a split without changing the underlying plan

1

Original

One four-container T01 line

2

Transform

Two two-container T01 lines with the same declaration

3

Predict

One extra line; unchanged container and portion totals

4

Check

Compare both representations with independent expectations

Line count may change while containers and planned portions stay equal.

Your implementation TODO

Supplied starting point

The notebook supplies BREAD-A and RICE-A line dictionaries.

You implement / do

Read each line, multiply containers by unit size, and accumulate container and portion totals outside/inside the loop correctly.

Check the evidence

Starter: 2 lines, 5 containers, 16 portions. Empty list → zeros. Split the bread line: total portions must stay 16.

Explain one field choice and trace one contribution into the total.

Show the component working

Step	What to show
1 · Locate	Identify the input, this component and its next user.
2 · Implement	Loop over lines; accumulate containers and portions.
3 · Check	Starter 2/5/16; empty → zeros; split line preserves total.
4 · Explain	Show your code, its actual result and why it behaves that way.

Explain one field choice and trace one contribution into the total.

Further reading in English

1. ## Python tutorial: data structures

   https://docs.python.org/3/tutorial/datastructures.html

2. ## Python tutorial: for statements

   https://docs.python.org/3/tutorial/controlflow.html#for-statements

Read one relevant section, then bring an example and a question.